

Lesson 8 - Quick Reference Card

▣ Basic Validation

```
// In Controller
$validated = $request->validate([
    'title' => 'required|max:255',
    'email' => 'required|email',
    'age' => 'required|integer|min:18',
]);

// Custom messages
$request->validate(..., [
    'title.required' => 'Title is required',
]);
```

▣ Validation Rules

String

```
'name' => 'required|string|min:3|max:255'
'username' => 'alpha_num'
'code' => 'alpha_dash'
'url' => 'url'
```

Numbers

```
'age' => 'integer|min:18|max:100'
'price' => 'numeric|min:0'
'quantity' => 'digits:4'
'amount' => 'between:10,1000'
```

Date

```
'birth_date' => 'date|before:today'
'start_date' => 'date|after:today'
```

```
'end_date' => 'date|after:start_date'
```

Email

```
'email' => 'required|email'  
'email' => 'email:rfc,dns'
```

Database

```
'email' => 'unique:users,email'  
'email' => 'unique:users,email,' . $userId  
'category_id' => 'exists:categories,id'
```

Boolean

```
'agree' => 'required|accepted'  
'active' => 'boolean'
```

Files

```
'avatar' => 'image|max:2048'  
'avatar' => 'image|mimes:jpeg,png,jpg'  
'document' => 'file|mimes:pdf,doc,docx|max:10240'  
'avatar' => 'dimensions:min_width=100,min_height=100'
```

Advanced

```
'status' => 'in:pending,approved,rejected'  
'username' => 'not_in:admin,root'  
'password_confirmation' => 'same:password'  
'new_password' => 'different:old_password'  
'phone' => 'required_if:contact_method,phone'  
'email' => 'required_unless:contact_method,phone'  
'last_name' => 'required_with:first_name'  
'phone' => 'required_without:email'
```

Form Request Classes

Create

```
php artisan make:request StorePostRequest
```

Code

```
class StorePostRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true; // or auth()->check()
    }

    public function rules(): array
    {
        return [
            'title' => 'required|string|max:255',
            'content' => 'required|string',
        ];
    }

    public function messages(): array
    {
        return [
            'title.required' => 'Title is required',
        ];
    }

    public function attributes(): array
    {
        return [
            'title' => 'post title',
        ];
    }
}
```

Usage

```
public function store(StorePostRequest $request)
{
    $validated = $request->validated();
    Post::create($validated);
}
```

```
}
```

▣ Custom Rules

Create

```
php artisan make:rule Uppercase
```

Code

```
class Uppercase implements ValidationRule
{
    public function validate(string $attribute, mixed $value, Closure $fail): void
    {
        if (strtoupper($value) !== $value) {
            $fail('Field must be uppercase');
        }
    }
}
```

Usage

```
use App\Rules\Uppercase;

'code' => ['required', new Uppercase],
```

▣ Array Validation

```
// Simple array
'tags' => 'required|array|min:1|max:5',
'tags.*' => 'string|max:50',

// Nested array
'products' => 'required|array',
'products.*.name' => 'required|string',
'products.*.price' => 'required|numeric',
```

```
'products.*.images' => 'nullable|array',  
'products.*.images.*' => 'image|max:2048',
```

□ Conditional Validation

```
public function rules(): array  
{  
    $rules = ['title' => 'required'];  
  
    if ($this->input('type') === 'video') {  
        $rules['video_url'] = 'required|url';  
    }  
  
    if ($this->isMethod('PUT')) {  
        $rules['email'] = 'unique:users,email,' . $this->route('user');  
    }  
  
    return $rules;  
}
```

□ Display Errors

All errors

```
@if ($errors->any())  
    <ul>  
        @foreach ($errors->all() as $error)  
            <li>{{ $error }}</li>  
        @endforeach  
    </ul>  
@endif
```

Specific field

```
@error('title')  
    <div>{{ $message }}</div>
```

```
@enderror
```

In Input

```
<input name="title" value="{{ old('title') }}" class="@error('title') is-invalid @enderror">
```

📄 Quick Examples

User Registration

```
$request->validate([
    'name' => 'required|string|max:255',
    'email' => 'required|email|unique:users',
    'password' => 'required|min:8|confirmed',
    'avatar' => 'nullable|image|max:2048',
]);
```

Create Post

```
$request->validate([
    'title' => 'required|max:255',
    'content' => 'required|min:100',
    'category_id' => 'required|exists:categories,id',
    'tags' => 'array|max:5',
    'tags.*' => 'exists:tags,id',
]);
```

File Upload

```
$request->validate([
    'photos' => 'required|array|min:1|max:5',
    'photos.*' => 'image|max:2048',
]);
```

▣ Best Practices

- ▣ Use Form Requests
 - ▣ Use `validated()` only
 - ▣ Clear user messages
 - ▣ Use `old()` in forms
 - ▣ Custom Rules for complex logic
 - ▣ Don't use `all()` instead of `validated()`
 - ▣ Don't forget `authorize()`
 - ▣ Don't use weak rules
-

▣ Quick Links

- [Main Lesson](#)
- [Previous Lesson](#)
- [Next Lesson](#)